

Lab Report - Homework 12

Student Name: Enis Ogdu

Course: Dynamic Documents and Front-end Web Development

Assignment: Homework 12 - Node.js and Express.js Introduction

Date: December 28, 2025

Assignment Overview

This lab report documents the setup and configuration of a Node.js development environment on macOS, including the creation of two projects:

1. **Part 1:** A basic Node.js project using the `serve` package to run a simple HTTP server
2. **Part 2:** An Express.js application generated using `express-generator` with Twig as the template engine

The exercises demonstrate fundamental Node.js concepts including package management with npm, project initialization, dependency installation, and running local development servers.

Part 1: Basic Node.js Project Setup

Environment Preparation

Node.js was initially not installed on the macOS system used for this exercise. To establish the required development environment, the Node.js Long-Term Support (LTS) version was downloaded and installed using the official installer available at nodejs.org.

Following the installation process, verification was performed using the macOS Terminal application. The following commands were executed to confirm the successful installation of both Node.js and npm (Node Package Manager):

```
node -v  
npm -v
```

The terminal returned version numbers for both Node.js and npm, confirming that the installation was completed successfully. The presence of these version numbers indicates that the Node.js runtime environment and the npm package manager are properly configured and ready for use in subsequent development tasks.

This section documents the initial setup of the Node.js development environment for the beginner-level exercise.

Project Directory Creation

To organize the project files, a dedicated directory was created on the Desktop. The following commands were executed in the Terminal:

```
cd Desktop
mkdir new
cd new
pwd
```

Result:

A new project directory named new was successfully created and set as the working directory.

npm Project Initialization

To initialize a new Node.js project, the npm package manager was used with the -y flag to automatically accept default configuration values. The following command was executed:

```
npm init -y
```

Created file:

- package.json

Content of package.json:

```
{
  "name": "new",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

The package.json file serves as the project manifest, containing metadata about the project and its dependencies. The initialization process successfully created this file with default values, establishing the foundation for the Node.js project.

Installing HTTP Server

To enable local web server functionality, a simple HTTP server package was installed using npm. The following command was executed:

```
npm install serve
```

Explanation:

A simple HTTP server package was installed using npm. It allows serving the project files over a local web server.

Created items:

- node_modules folder
- package-lock.json

The node_modules folder contains all installed dependencies and their sub-dependencies, while package-lock.json records the exact version tree of packages installed, ensuring consistent installations across different environments.

Adding npm start Script

To simplify the process of starting the local development server, a custom npm script was added to the package.json file. The scripts section was modified as follows:

```
"scripts": {  
  "start": "serve public -l 5000"  
}
```

Explanation:

This script allows starting the local server by running `npm start` in the terminal. It serves the project files at `http://localhost:5000`.

The script configuration specifies that the serve package should serve files from the public directory and listen on port 5000. This provides a convenient way to launch the development server with a single command, rather than typing the full server command each time.

Starting the Server and Testing

To verify that the development environment was properly configured, the local server was started using the npm script. The following command was executed:

```
npm start
```

Result:

```
> new@1.0.0 start  
> serve public -l 5000
```

Serving!

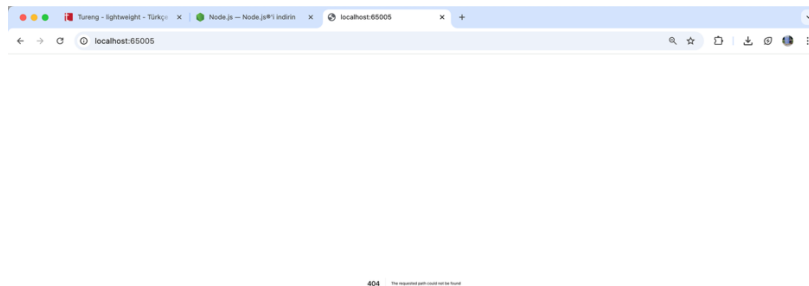
- Local: `http://localhost:65005`
- Network: `http://192.168.0.112:65005`

Note: The server automatically selected port 65005 because port 5000 was already in use on the system.

Verification Screenshots

Two screenshots were captured to document the successful server operation:

1. Browser View:



Browser showing the project being served at localhost:65005

The browser screenshot shows the server responding at `http://localhost:65005`. The 404 error is expected at this stage, as no files have been created in the public directory yet.

3. Terminal Output:

```
new — node < npm start TMPDIR=/var/folders/hs/pdrqwnyx4f7_5gbg3_zp...

Serving!

- Local:    http://localhost:65005
- Network:  http://192.168.0.112:65005

This port was picked because 5000 is in use.

Copied local address to clipboard!

HTTP 12/28/2025 6:38:33 PM ::1 GET /
HTTP 12/28/2025 6:38:33 PM ::1 Returned 404 in 32 ms
HTTP 12/28/2025 6:38:33 PM ::1 GET /favicon.ico
HTTP 12/28/2025 6:38:33 PM ::1 Returned 404 in 2 ms
HTTP 12/28/2025 6:39:07 PM ::1 GET /
HTTP 12/28/2025 6:39:07 PM ::1 Returned 404 in 2 ms
HTTP 12/28/2025 6:39:07 PM ::1 GET /
HTTP 12/28/2025 6:39:07 PM ::1 Returned 404 in 2 ms
```

Terminal showing npm start running successfully

The terminal screenshot confirms that the `npm start` command executed successfully, displaying the “Serving!” message along with the local and network addresses. The HTTP request logs show the server is actively responding to browser requests.

These verification steps confirm that the Node.js development environment has been successfully set up and the local web server is functioning correctly.

Part 2: An Elementary Express Project

Installing Express Generator Globally

To create a structured Express.js application quickly, the Express application generator tool was installed globally on the system. The following command was executed:

```
npm install -g express-generator
```

Explanation:

Express-generator is a tool that creates a ready-made Express.js project. Installing it globally allows you to run the `express` command anywhere on your computer.

The `-g` flag installs the package globally, making the `express` command available system-wide rather than only in a specific project directory. This enables rapid scaffolding of new Express applications with a standardized project structure.

Creating Express Project with Twig View Engine

A new Express.js project was created in a separate directory using the `express-generator` tool. The following commands were executed:

```
cd ~/Desktop
express express_project --view=twig
cd express_project
```

Explanation:

Express-generator created a new Express.js project in the folder `express_project` and set Twig as the template engine. All necessary files and folder structure are ready.

The `--view=twig` parameter specifies Twig as the template engine instead of the default view engine. This generates a project configured to use Twig templates for rendering dynamic HTML pages.

Created files/folders include:

- `app.js` - Main application file

- `package.json` - Project dependencies and scripts
- `routes/` - Route handlers directory
- `views/` - Twig templates directory
- `public/` - Static files directory (CSS, JavaScript, images)

This scaffolded structure provides a complete foundation for building a web application with Express.js, following best practices for project organization.

Installing Project Dependencies

Before running the Express application, all required dependencies needed to be installed. The following command was executed:

```
npm install
```

Explanation:

This command installs all dependencies required for the Express project to run. Without it, the server will not start.

The `npm install` command reads the `package.json` file and downloads all packages listed in the dependencies section. These packages include Express.js itself, the Twig template engine, and other necessary modules for the application to function properly.

Result:

- `node_modules/` folder created
- All required packages installed

The installation process creates a `node_modules` directory containing all the dependencies and their sub-dependencies, making the application ready to run.

Running the Express Server

With all dependencies installed, the Express application was ready to run. The following command was executed to start the development server:

```
npm start
```

Explanation:

- Express-generator created a new project with Twig templates
- Dependencies installed with `npm install`
- Server started with `npm start` and is accessible in the browser

The `npm start` command executes the start script defined in `package.json`, which typically runs `node ./bin/www`. This starts the Express server on the default port 3000.

Result:

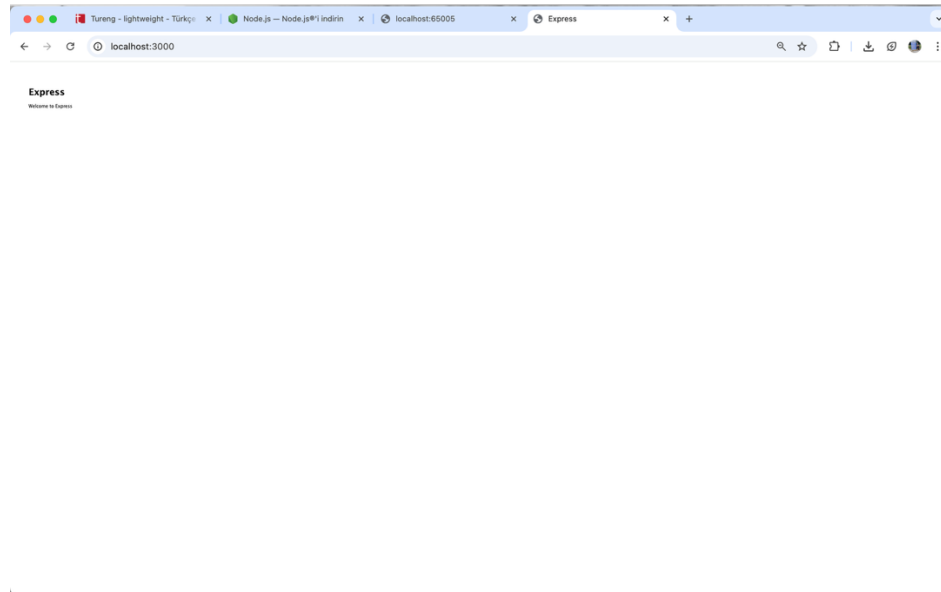
- Browser shows the default Express Welcome page

- Terminal shows `npm start` running without errors

Verification Screenshots

Two screenshots were captured to document the successful Express server operation:

1. Browser View - Express Welcome Page:



Browser displaying the Express Welcome page at localhost:3000

The browser screenshot shows the Express application successfully running at `http://localhost:3000`, displaying the default “Express - Welcome to Express” page. This confirms that the server is properly configured and responding to HTTP requests.

2. Terminal Output - Server Running:

```
express_project — node • npm start TMPDIR=/var/folders/hs/pdrqwnyx4f...
Password:
enisogdum@Eniss-MacBook-Air express_project % rm -rf node_modules
enisogdum@Eniss-MacBook-Air express_project % npm install

added 61 packages, and audited 62 packages in 5s

9 vulnerabilities (5 low, 4 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
enisogdum@Eniss-MacBook-Air express_project % npm start

> express-project@0.0.0 start
> node ./bin/www

GET / 200 20.869 ms - 207
GET /stylesheets/style.css 200 1.273 ms - 111
GET /favicon.ico 404 2.581 ms - 1258
```

Terminal showing npm start running successfully

The terminal screenshot shows the complete workflow: removing old `node_modules`, running `npm install` (which installed 61 packages), and successfully starting the server with `npm start`. The HTTP request logs (`GET / 200`, `GET /stylesheets/style.css 200`) confirm that the server is actively handling browser requests and serving both the HTML page and CSS stylesheet correctly.

These verification steps confirm that the Express.js application has been successfully created, configured with Twig as the template engine, and is running properly on the local development server.